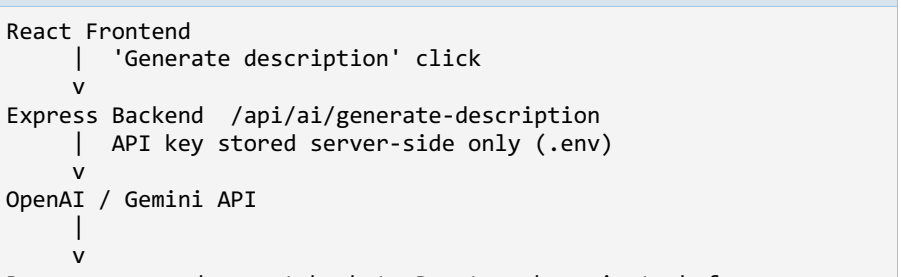
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

ADVANCED WEB DEVELOPMENT FRAMEWORKS (ITUE301)

Practical 13: AI API Integration into a Web Application

CO/PO	CO6 / PO3, PO5, PO12
Objective	To integrate an external AI service into the existing application and understand responsible AI usage.
Prerequisites	• Practical 6 completed working full-stack Task Management application • An active OpenAI API key or Gemini API key (institution-provided or free-tier)
Theory Concepts (Self-Study Reference)	
Coursera Pointer	No dedicated Coursera module refer to the YouTube video 'Build an AI Image Generator with OpenAI & Node.js' by Traversy Media for general OpenAI API integration patterns in Node.js (adapt the API call pattern to a text-based use case for this practical).
Architecture / Diagram	
Diagram	
Problem Definition	
Problem Statement	• Add an AI-powered feature to the Task Management application using the OpenAI API or Gemini API. • The feature must serve a meaningful purpose for example, auto-generating a task description, suggesting a priority level, or proposing a deadline based on the task title. • The API key must be stored and used only on the backend (.env), never exposed to the frontend. • Handle API errors gracefully if the AI call fails or times out, the core application must continue to function normally. • Display the AI-generated response meaningfully in the React UI, not as raw JSON.
Key Questions / Analysis	• Why must the AI API key be called from the backend rather than directly from the React frontend? • What does 'graceful degradation' mean in this context, and why does it matter for a feature that depends on a third-party service outside your control? • What are the privacy and governance implications of sending task data to an external AI API, and what kind of data should never be sent this way?
Supplementary Problems	• Add a loading state on the frontend while waiting for the AI response.

	• Add a simple rate limit (e.g., max 1 AI request per task per minute) to avoid excessive API usage. • Display a clear disclaimer in the UI that the suggestion is AI-generated and should be reviewed by the user.
Key Skills Addressed	• Calling an external AI API securely from a Node/Express backend • Designing a feature that degrades gracefully when a third-party dependency fails • Parsing and displaying AI-generated content meaningfully in a React UI • Reasoning about ethical, privacy, and governance implications of AI integration
Learning Outcome	Students will be able to securely integrate a third-party AI API into a full-stack application, handle failures gracefully, and articulate the responsible-AI considerations involved.
	Lab Session Step-by-Step

Steps	1. Install the OpenAI SDK (or use fetch directly for Gemini):
	`npm install openai`
	2. Store the API key in .env (never commit this file):
	`OPENAI_API_KEY=your_key_here`
	3. Create a backend route that accepts a task title and returns a generated description:
	`const OpenAI = require('openai');`
	`const client = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });`
	`app.post('/api/ai/generate-description', async (req, res) => {`
	`  try {`
	`  const completion = await client.chat.completions.create({`
	`  model: 'gpt-4o-mini',`
	`  messages: [{ role: 'user', content: `Write a short task description for: ${req.body.title}` }]`
	`  });`
	`  res.json({ description: completion.choices[0].message.content });`
	`  } catch (err) {`
	`  res.status(200).json({ description: '', fallback: true });`
	`  }`
	`});`
	4. On the frontend, add a "Generate description" button on the task creation form that calls this endpoint.
	5. Display the returned description as an editable suggestion the student can accept or modify.
	6. Test the failure path by temporarily using an invalid API key, and confirm the app shows a fallback message instead of crashing.

Tools / Technology / Resources Required	• Node.js (v18+), npm • OpenAI API or Gemini API (institution-provided key) • Express.js, React 18+ • Postman (for backend endpoint testing)
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours
	Faculty Teaching Guide
Guidance Notes	• Distribute the institution-sponsored API key/access method before the session begins do not let key provisioning eat into lab time. • Emphasize the security point strongly: the API key must never appear in any frontend code or be committed to GitHub. • Use this session to also briefly discuss Responsible AI from the syllabus what data should and should not be sent to a third-party model.

	• Demonstrate the graceful-degradation failure path live (using a broken key) so students see it's a designed behavior, not a bug.
Common Mistakes	• Calling the OpenAI/Gemini API directly from the React frontend, exposing the API key publicly in browser network requests. • Not handling API errors, causing the entire task creation flow to break if the AI call fails. • Committing the .env file with the real API key to GitHub. • Treating the AI response as the final answer rather than an editable suggestion, removing user control.

Troubleshooting Guide

Symptom	Likely Cause	Fix
401 Unauthorized from OpenAI/Gemini	Invalid or missing API key	Verify the key in .env is correct and the variable name matches what the code reads
API key visible in browser DevTools Network tab	AI API called directly from frontend instead of through the backend	Move the AI API call entirely to an Express route; frontend should only call your own backend endpoint
App crashes when AI call fails	No try/catch around the API call	Wrap the API call in try/catch and return a fallback response instead of letting the error propagate
Rate limit / quota exceeded error	Too many requests sent in a short time, especially across a full lab batch	Add a basic per-task rate limit and inform students in advance to avoid repeated unnecessary calls

Student Assignment / Post Laboratory Work

Tasks	• Review the Theory Concepts section and the AI integration reference video before next week. • Add a loading state on the frontend while waiting for the AI response. • Add a visible disclaimer noting the content is AI-generated and should be reviewed.

GitHub Deliverables

Expected Repository Contents	• Same repository as Practical 6, updated with new commits • Working AI-powered feature with graceful failure handling • .env excluded via .gitignore; no API key committed anywhere in history • At least one commit specifically for AI integration

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
Conceptual understanding (2 Marks)	Explains what and why, not just what was done
Implementation (4 Marks)	Code written, runs, and is original
Correctness (2 Marks)	Output matches expected result
Reflection (1 marks)	Errors documented and explained
Lab file evidence (1 marks)	Completeness and presentation
Total: 10 marks	**Passing Threshold: 5 / 10 (50%)**